

```
In [1]: import pandas as pd                # Step 1: Import Libraries:
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [3]: df = pd.read_csv('C:/Users/hp/Desktop/financial_data.csv') #Step 2: Load the D
print(df.head())
```

	Date	Open	High	Low	Close	Volume
0	2023-01-01	150.25	153.00	149.50	152.00	1000000
1	2023-01-02	153.50	155.75	152.00	154.00	1200000
2	2023-01-03	154.75	156.50	153.25	155.25	1150000
3	2023-01-04	156.00	157.00	154.00	156.25	1300000
4	2023-01-05	157.25	158.50	156.00	157.00	1250000

```
In [4]: df['Date'] = pd.to_datetime(df['Date']) #Step 3: Data Preprocessing: # Convert
```

```
In [5]: print(df.isnull().sum()) # Check for missing values (if any)
```

```
Date      0
Open      0
High      0
Low       0
Close     0
Volume    0
dtype: int64
```

```
In [6]: X = df[['Open', 'High', 'Low', 'Volume']] # Features (Open, High, Low, Volume)
y = df['Close']
```

```
In [12]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
print(f"Training set size: {len(X_train)}")
print(f"Test set size: {len(X_test)}")
```

```
Training set size: 4
Test set size: 1
```

```
In [11]: scaler = StandardScaler()                #Step 5: Feature Scaling:
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [13]: model = LinearRegression() #Step 6: Train the Model (Linear Regression):Initia
```

```
In [14]: model.fit(X_train_scaled, y_train) #Train the model:
```

```
Out[14]: ▼ LinearRegression ⓘ ?
LinearRegression()
```

```
In [15]: y_pred = model.predict(X_test_scaled) # Make predictions:
```

```
In [17]: mse = mean_squared_error(y_test, y_pred)      #Step 7: Evaluate the Model:# Evalu
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")
```

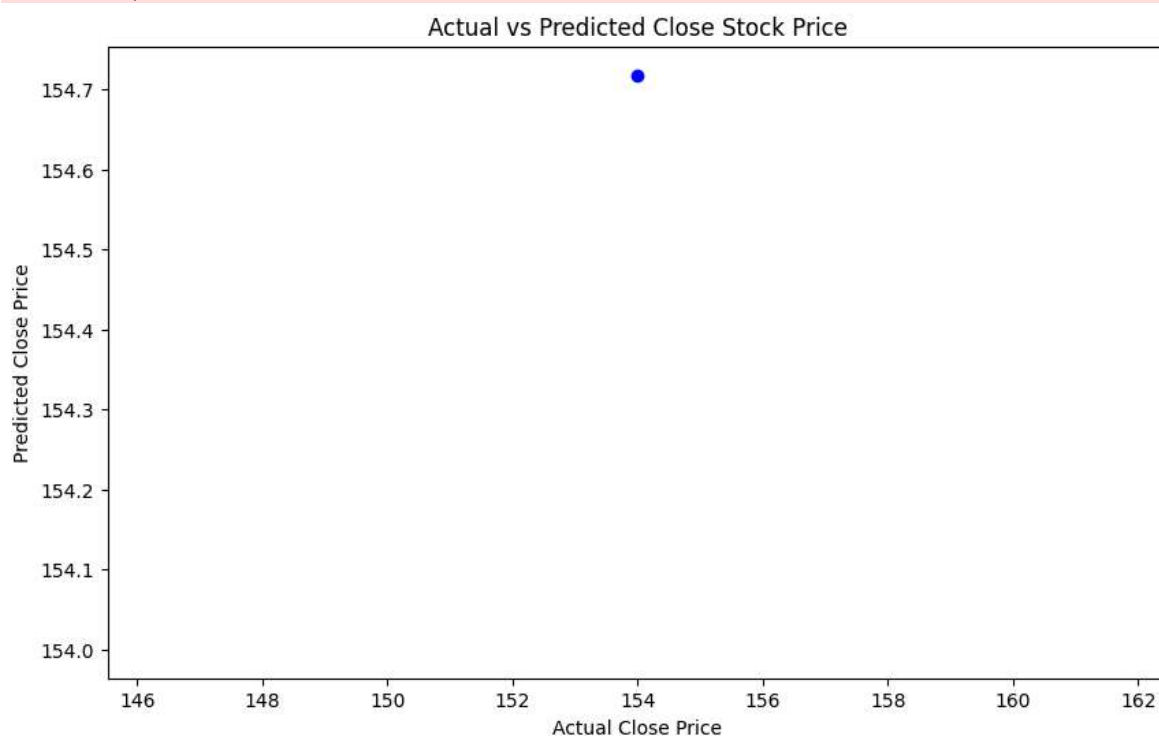
Mean Squared Error: 0.5153276452888227

R-squared: nan

C:\Users\hp\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics_regression.py:1266: UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.
warnings.warn(msg, UndefinedMetricWarning)

```
In [18]: plt.figure(figsize=(10, 6))                  #Step 8: Visualize Actual vs Predict
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--k', color='red')
plt.xlabel('Actual Close Price')
plt.ylabel('Predicted Close Price')
plt.title('Actual vs Predicted Close Stock Price')
plt.show()
```

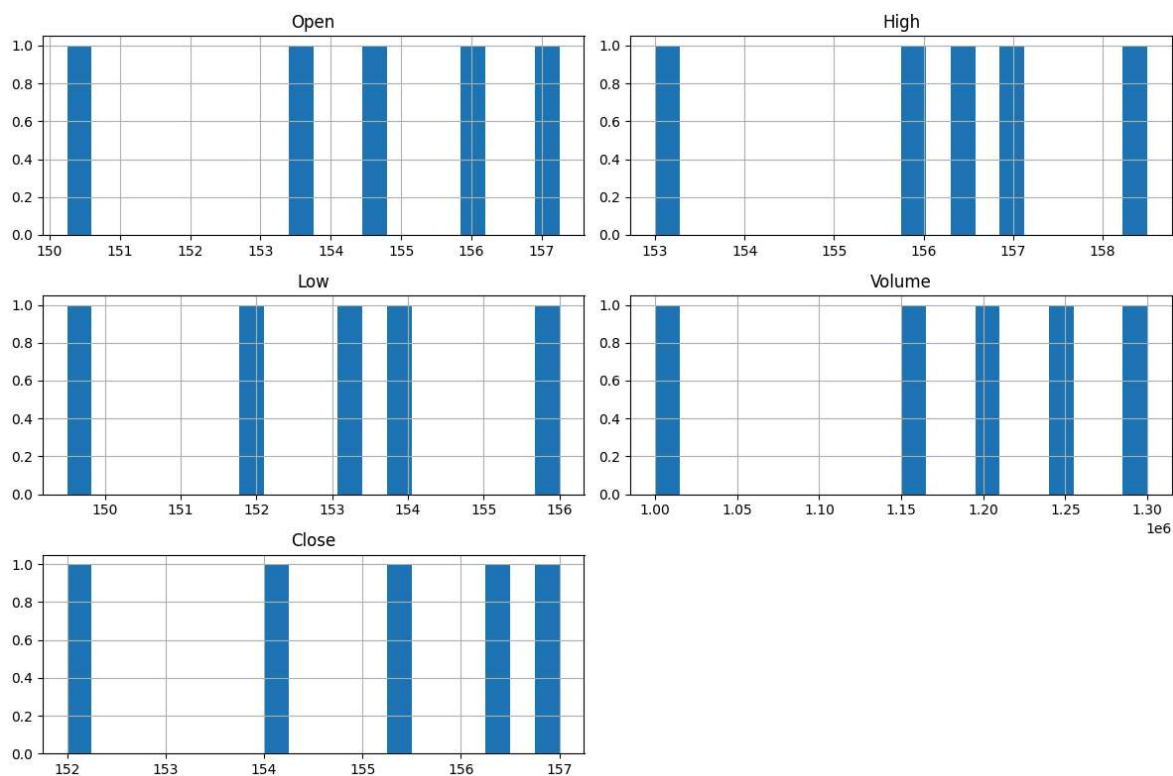
C:\Users\hp\AppData\Local\Temp\ipykernel_9312\2070597881.py:3: UserWarning: color is redundantly defined by the 'color' keyword argument and the fmt string "--k" (-> color='k'). The keyword argument will take precedence.
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--k', color='red')



```
In [20]: plt.figure(figsize=(8, 6))                  #Step 9: Visualize Feature Correlat
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.show()
```



```
In [21]: df[['Open', 'High', 'Low', 'Volume', 'Close']].hist(bins=20, figsize=(12, 8))
plt.tight_layout()
plt.show()
```



```
In [ ]:
```